

Scikit-Learn Handout & Reference Guide

1. The Core Philosophy & Workflow

`scikit-learn` (`sklearn`) is Python's gold standard for machine learning. Its power lies in a simple, consistent API. Every model follows the same pattern, allowing you to easily swap models and experiment.

The 5-Step Supervised Learning "Recipe"

This is the workflow you will use for almost every regression and classification problem.

1. Prepare Data (**X**, **y**)

- **X**: The **features** (your "input variables"). Must be a 2D numeric array (e.g., a pandas DataFrame).
- **y**: The **target** (your "answer"). A 1D array or Series (e.g., 'price' or 'is_spam').

2. Split Data

- Split your data into `train` and `test` sets to get an honest evaluation.
- `from sklearn.model_selection import train_test_split`
- `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)`

3. Instantiate & Fit Model

- `from sklearn.linear_model import LinearRegression`
- `model = LinearRegression()`: Create the "machine."
- `model.fit(X_train, y_train)`: The "learning" step.

4. Predict

- `predictions = model.predict(X_test)`: Use the *trained* model on *unseen* data.

5. Evaluate

- `from sklearn.metrics import r2_score`
- `score = r2_score(y_test, predictions)`: Compare the model's guesses to the real answers.

2. Essential Tools & Best Practices

Data Preprocessing (`StandardScaler`)

- **What?** A tool that rescales all your features to have a mean of 0 and a standard deviation of 1.

- **Why?** Many models (like `LinearRegression`, `k-NN`, and `KMeans`) are highly sensitive to features with different scales (e.g., "Age" 18-70 vs. "Salary" 30k-200k). Scaling puts all features on a level playing field.

How (The *Wrong* Way):

```
# PROBLEM: This causes DATA LEAKAGE
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# This scaler "learned" from the test set. Your scores will be fake.
```

How (The *Right* Way):

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()

# 1. Fit *ONLY* on the training data
X_train_scaled = scaler.fit_transform(X_train)

# 2. Transform the test data (using the mean/std from train)
X_test_scaled = scaler.transform(X_test)
```

Pipelines: The "Pro" Workflow

- **What?** A `Pipeline` object chains all your steps (e.g., scaling + modeling) into one.
- **Why?** It's cleaner, easier, and **automatically prevents data leakage**. It handles all the `fit_transform` and `transform` logic for you, even during cross-validation.

How:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression

# Create the pipeline as a list of ('name', object) steps
pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LinearRegression())
])

# The pipeline does everything for you!
pipe.fit(X_train, y_train)
predictions = pipe.predict(X_test)
score = pipe.score(X_test, y_test) # Uses the model's default metric (R^2)
```

Robust Evaluation (**cross_val_score**)

- **What?** A single **train_test_split** can be "lucky" or "unlucky." Cross-validation is a more robust way to score your model. It splits the data into 5 (or 'k') "folds," then trains and tests 5 times, averaging the result.

How:

```
from sklearn.model_selection import cross_val_score

# Use your full pipeline and all your data (X, y)
# cv=5 means 5-fold cross-validation
scores = cross_val_score(pipe, X, y, cv=5, scoring='r2')

print(scores)
# [0.85, 0.82, 0.88, 0.81, 0.86]
print(f"Average R^2: {scores.mean():.3f}")
```

3. Regression (Predicting a Number)

- **Goal:** Predict a continuous, numeric value (e.g., price, temperature, score).

Common Models

- **LinearRegression:** The baseline. Fits a straight line to the data.
 - **model.coef_:** The coefficients (slope) for each feature.
 - **model.intercept_:** The intercept (baseline) where the line crosses the y-axis.
- **Ridge & Lasso:** Regularized regression models. They are more complex and help prevent **overfitting** by "punishing" large coefficients. Use **Lasso** if you suspect many features are useless.

Key Evaluation Metrics (**sklearn.metrics**)

- **r2_score (R-squared):** "What percentage of the variance in **y** is explained by **X**?"
 - **1.0** is a perfect fit. **0.0** is no better than guessing the average.
- **mean_squared_error (MSE):** The average of the *squared* errors.
 - **Punishes large errors** heavily. Lower is better.
- **mean_absolute_error (MAE):** The average of the *absolute* errors.
 - **Easy to interpret** (e.g., "On average, our prediction was off by \$X units"). Lower is better.

4. Classification (Predicting a Category)

- **Goal:** Predict a discrete label or class (e.g., **Spam/Not Spam**, **Pass/Fail**, **Type A/B/C**).

Common Models

- **KNeighborsClassifier (k-NN)**: Classifies a point based on a "majority vote" of its **k** nearest neighbors.
 - **Warning**: Is *not* "smart." It's lazy and highly sensitive to feature scaling.
- **LogisticRegression**: Despite the name, it's a classification model. It predicts the *probability* (from 0.0 to 1.0) that a sample belongs to a class.
- **DecisionTreeClassifier**: A flow-chart of "if-then" rules. Very interpretable and (unlike the others) does *not* require feature scaling.

Key Evaluation Metrics (**sklearn.metrics**)

- **accuracy_score**: (**Correct / Total**). Simple, but **very misleading** on imbalanced data.
- **confusion_matrix**: A table showing your model's exact mistakes.

	Predicted: NO	Predicted: YES
Actual: NO	True Negative (TN) (Correctly said "No")	False Positive (FP) (Wrongly said "Yes", Type I Error)
Actual: YES	False Negative (FN) (Wrongly said "No", Type II Error)	True Positive (TP) (Correctly said "Yes")

- **precision_score**: $TP / (TP + FP)$
 - **The Question**: "Of all the times the model predicted **YES**, how often was it correct?"
 - **Analogy**: A spam filter. You want **High Precision**. You must be "precise" and avoid **False Positives** (don't send a good email to spam).
- **recall_score**: $TP / (TP + FN)$

- **The Question:** "Of all the *actual* **YES** cases, how many did the model find (recall)?"
- **Analogy:** A medical test. You want **High Recall**. You must "recall" all sick patients and avoid **False Negatives** (don't tell a sick person they are healthy).
- **f1_score:** The harmonic mean (balanced average) of Precision and Recall. A great single-number summary for imbalanced classes.

5. Unsupervised Learning (Finding Groups)

Classification vs. Clustering: A Key Distinction

It's easy to confuse Classification and Clustering, as both involve "groups." Here is the critical difference:

Concept	Classification	Clustering
Learning Type	Supervised Learning	Unsupervised Learning
The Goal	Predict a <i>known</i> category	Discover <i>unknown</i> groups
Data Used	Features (X) and Target Labels (y)	Features (X) only
Example Question	"Based on this email's text, is it 'Spam' or 'Not Spam'?"	"Based on these customers' purchase habits, can we find 3 distinct segments?"

In short, Classification *predicts* a label you already have. Clustering *finds* labels you don't.

- **Goal:** Find hidden patterns or groups in the data when you have **NO y target**.

Common Model: **KMeans** Clustering

- **What?** An algorithm that partitions data into **k** clusters.
- **How?** It iteratively (1) **Assigns** each point to the nearest cluster center (centroid) and (2) **Updates** the centroid to the mean of its assigned points.
- **Warning:** You must **StandardScale** your data first!

Finding the Right Number of Clusters (k)

- `kmeans.inertia_`: The **Within-Cluster Sum of Squares (WCSS)**. A measure of how "tight" the clusters are (lower is better).
- **Elbow Method**:
 - Run `KMeans` for `k=1` through `k=10`.
 - Plot `k` (x-axis) vs. `inertia_` (y-axis).
 - Look for the "elbow": the point of **diminishing returns**, where adding another cluster stops reducing the inertia by a significant amount.
- **Silhouette Score (`silhouette_score`)**:
 - A "better" metric than inertia. Measures both cluster **tightness** (how close points are to their own cluster) and **separation** (how far they are from other clusters).
 - Score ranges from **-1 (bad)** to **+1 (good)**. **0** means overlapping clusters. Unlike the elbow method, here **higher is better**.

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Elbow Method
inertia_values = []
k_range = range(1, 11)
for k in k_range:
    kmeans = KMeans(n_clusters=k, n_init=10, random_state=42)
    kmeans.fit(X_scaled)
    inertia_values.append(kmeans.inertia_)

# plt.plot(k_range, inertia_values, marker='o')
# Plotting code from previous example

# Silhouette Score (run this in the same loop)
# Note: Silhouette score can't be computed for k=1
silhouette_scores = []
k_range_sil = range(2, 11) # Start from 2 clusters
for k in k_range_sil:
    kmeans = KMeans(n_clusters=k, n_init=10, random_state=42)
    labels = kmeans.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    silhouette_scores.append(score)

# plt.plot(k_range_sil, silhouette_scores, marker='o')
```

6. Core Concepts Glossary

- **Overfitting:** The model "memorized" the *noise* in the training data. It gets a great score on `X_train` but a terrible score on `X_test`.
- **Underfitting:** The model is *too simple* to capture the underlying pattern. It gets a terrible score on *both* `X_train` and `X_test`.
- **Bias-Variance Trade-off:**
 - **High Bias:** (Underfitting). The model is too simple and makes strong, "biased" assumptions. (e.g., a `LinearRegression` model on curved data, or a large `k` in k-NN).
 - **High Variance:** (Overfitting). The model is too complex and "nervous." Its predictions change wildly based on small changes in the training data. (e.g., a `DecisionTree` that is too deep, or `k=1` in k-NN).
- **Data Leakage:** The *cardinal sin* of machine learning. When information from your test set (or future) "leaks" into your training process. This gives a fake, optimistically high score.
 - **Example 1:** Fitting your `StandardScaler` on `X` *before* the `train_test_split`.
 - **Example 2:** Including a feature like "total_lifetime_value" when you are trying to predict "will_sign_up".